

# SafeVault - Phase 2 Sprint 1 Deliverable

---

**Repositorio:** desofs2026\_thu\_ffs\_3

**Turma:** thu\_ffs

**Sprint:** Phase 2 - Sprint 1

**Data:** 2026-05-18

---

## 1. Objetivo da Sprint

Implementar e automatizar o conjunto de praticas de desenvolvimento e testes de seguranca exigidas na rubrica 6.2 (Development, Build and Test, Pipeline automation, ASVS), mantendo coerencia entre documentacao e desenvolvimento.

---

## 2. Resumo do que foi feito

- Pipeline CI com build, testes e SCA.
  - SAST via CodeQL.
  - DAST baseline via OWASP ZAP (workflow manual para alvo externo).
  - IAST leve com middleware de deteccao de padroes suspeitos.
  - Hardening tecnico: segredos removidos do repositorio, CSRF dinamico, erros sem detalhe interno, validacao de magic bytes.
  - Documentacao completa e rastreabilidade actualizada.
- 

## 3. Desenvolvimento (30%)

### 3.1 Boas praticas e code review

- Code review formalizado com CODEOWNERS e template de PR:
  - [.github/CODEOWNERS](#)
  - [.github/pull\\_request\\_template.md](#)
- Politica de revisao: pelo menos 1 revisor e checklist de seguranca no PR.

### 3.2 Hardening tecnico implementado

- Segredos removidos de ficheiros versionados:
  - [src/InterfaceAdapters/appsettings.json](#)
  - [src/InterfaceAdapters/appsettings.Development.json](#)
- Validacao de segredo e connection string em runtime:
  - [src/InterfaceAdapters/Program.cs](#)
- CSRF token dinamico com validade de 30 minutos:
  - [src/Infrastructure/Security/CsrfTokenService.cs](#)
  - [src/InterfaceAdapters/Middleware/CsrfTokenMiddleware.cs](#)

- [src/InterfaceAdapters/Controllers/AuthController.cs](#)
- Erros sem detalhe interno, com correlation id:
  - [src/InterfaceAdapters/Middleware/ExceptionHandlingMiddleware.cs](#)
- Validacao de magic bytes para uploads:
  - [src/Application/Services/DocumentService.cs](#)

### 3.3 IAST leve (runtime detection)

- Middleware de deteccao de padroes suspeitos (SQLi/XSS/path traversal) em runtime:
    - [src/InterfaceAdapters/Middleware/lastMonitoringMiddleware.cs](#)
  - Controlado por configuracao:
    - [src/InterfaceAdapters/appsettings.json](#)
    - [src/InterfaceAdapters/appsettings.Development.json](#)
- 

## 4. Build and Test (30%)

### 4.1 Execucao automatizada de testes

- Pipeline executa build e testes com cobertura:
  - [.github/workflows/ci.yml](#)
- Testes existentes:
  - [tests/ApplicationTests](#)
  - [tests/InfrastructureTests](#)
  - [tests/InterfaceAdaptersTests](#)

### 4.2 Validacao de configuracao

- Falha rapida se secrets/connection string estiverem ausentes:
  - [src/InterfaceAdapters/Program.cs](#)

### 4.3 Analise dinamica (DAST)

- OWASP ZAP baseline via workflow manual para alvo externo (staging):
    - [.github/workflows/dast.yml](#)
    - [.zap/rules.tsv](#)
- 

## 5. Pipeline automation (20%)

### 5.1 CI principal

- Build, testes e SCA automatizados em cada push/PR:
  - [.github/workflows/ci.yml](#)

### 5.2 SAST

- CodeQL configurado e automatizado:
  - [.github/workflows/codeql.yml](#)

## 5.3 SCA

- dotnet list package --vulnerable automatizado com artefacto:
  - [.github/workflows/ci.yml](#)

## 5.4 DAST

- OWASP ZAP baseline para alvo configuravel via workflow\_dispatch:
    - [.github/workflows/dast.yml](#)
- 

## 6. ASVS (15%)

Checklist actualizado e alinhado com evidencias reais:

- [phase2\\_sprint1\\_asvs\\_checklist.md](#)
- 

## 7. Comandos e reproducao local

### 7.1 Variaveis de ambiente obrigatorias

```
ConnectionStrings__DefaultConnection=Host=localhost;Port=5432;Database=safevault;U
sername=postgres;Password=postgres
Jwt__SigningKey=CHANGE_ME_WITH_AT_LEAST_32_CHARACTERS
Jwt__Issuer=SafeVault
Jwt__Audience=SafeVault.Api
```

### 7.2 Build e testes

```
dotnet restore
dotnet build --configuration Release
dotnet test --configuration Release
```

### 7.3 DAST manual (ZAP)

Executar o workflow [dast.yml](#) via GitHub Actions e fornecer um URL de staging valido (HTTPS).

---

## 8. Rastreabilidade

- Matriz de rastreabilidade Sprint 1:
    - [phase2\\_sprint1\\_traceability\\_matrix.md](#)
- 

## 9. Lacunas conhecidas e proxima sprint

- Integrar DAST automatico contra ambiente de staging com dados reais.

- Estender IAST com instrumentacao mais profunda (body inspection controlado e correlacao request->sink).
- Completar evidencias de execucao (links diretos para runs da pipeline).